SAMPLE: illustrative engagement; not a client. Every number below comes from real runs on a local test box. The targets are a reference implementation and a mock that was deliberately made vulnerable, both on loopback. No client system was tested.

CIRIGLIANO SECURITY · AI AGENT SECURITY REVIEW

AI Agent Security Review: Sample Report

An engineering-focused assessment of agent permissions and execution controls.

Engagement	SAMPLE-DEMO-001 (illustrative)
Reviewer	Greg Cirigliano, Cirigliano Security
Test window	Fri Oct 23, 2026, 12:53–12:55 EDT (single session)
Targets	**A**: a reference fail-closed custody implementation, through its local test adapter · **B**: the harness's mock agent service in 7 configurations
Tooling	agent test harness (tests 3–8, hash-chained evidence ledger), promptfoo red-team smoke

What this report does and does not show

WHAT IT SHOWS

- How a review is run, and the evidence it produces: revocation, replay, approval-gate, decision-reason and audit-ledger tests, each with a positive control and a tamper-evident ledger.
- That the method detects each of the 6 flaw types deliberately injected into a mock service.
- How one reference implementation behaved on the tests that applied to it.

WHAT IT DOES NOT SHOW

- That any real system is secure. Two areas could not be tested on the reference implementation, and the prompt-level run used a scripted stub, so it says nothing about any real model.
- Real-world effectiveness. Timings are local loopback wall-clock times, not a production SLA.
- Flaws in any real product. Target B is deliberately vulnerable.

A technically correct result is not comprehensive AI-agent security assurance. This sample proves the method, not real-world effectiveness. Broader LLM red teaming is an additional service, not the focus of this review.

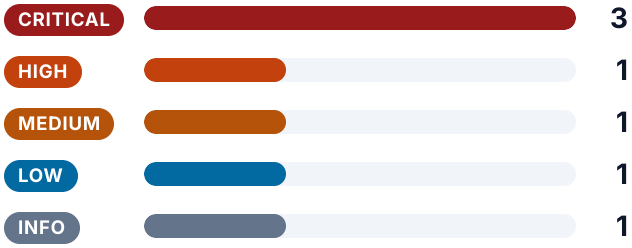
Executive overview

HEADLINE RESULT

On the reference implementation, every control the test interface could exercise held: no findings. The method also caught all 6 flaws deliberately built into a mock service.

This is a demonstration of the review method on local test systems. It is not a rating of any real product.

FINDINGS BY SEVERITY



6 findings (F-01 to F-06) plus 1 note. F-01 to F-06 are on Target B (injected flaws). The Info note, I-01, is on Target A, which has no Critical, High, Medium or Low findings.

RESULTS AT A GLANCE

	TARGET A	TARGET B
What it is	Reference implementation	Mock service, injected flaws
Findings	0	6
Applicable tests passed	4 of 4	clean: 6 of 6
Calls that worked after revoke	0 of 31	F-04: 5 calls
Replays accepted	0 of 10	F-01 caught
Approval gates as expected	9 of 9	F-02 caught
Not applicable	2 tests	0
Injected flaws caught	n/a	6 of 6

Three key takeaways

1. **Revoked credentials stopped working at once on Target A.** 0 of 31 tool calls succeeded after revocation, the first was refused 3 ms after the revoke returned, and 0 of 10 replays of the captured credential were accepted.
2. **Approval gates and the audit trail held, and the evidence can be checked.** All 9 of 9 approval-gate outcomes matched expectations and the target's own ledger verified on 74 of 74 snapshots. Every run writes a hash-chained ledger; all 8 verify, and an edited entry and a deleted entry were both detected.
3. **The method finds real classes of control failure.** Each of the 6 injected flaws (3 Critical, 1 High, 1 Medium, 1 Low) produced exactly one failing test, while the clean configuration passed 6 of 6. The remediation roadmap (page 9) turns these into a fix order.

Honest limits, still in force: two tests (fail-closed on auth outage, rate-limit burst) were **not applicable** to Target A and are not counted as passes. The prompt-level red team ran only against a scripted stub and is **not validated against a real model**. Latency figures are local loopback times, not a production SLA.

Assessment coverage

Every cell below is taken from the results in Sections 4 and 5. Nothing is shown as passed unless it was run.

● Tested and passed ● Tested and failed ● Not applicable ○ Not tested or not validated against a real model

TEST AREA	TARGET A reference implementation	TARGET B deliberately vulnerable mock
Revoke latency / post-revoke use Test 3	✓ **Tested and passed** 0/31 post-revoke successes	✗ **Tested and failed** F-04 detected (flaw injected)
Fail-closed on auth outage Test 4	– **Not applicable** No adapter control	✗ **Tested and failed** F-03 detected (flaw injected)
Replay of captured credential Test 5	✓ **Tested and passed** 0/10 replays accepted	✗ **Tested and failed** F-01 detected (flaw injected)
Approval gates Test 6	✓ **Tested and passed** 9/9 as expected	✗ **Tested and failed** F-02 detected (flaw injected)
Rate limit (burst) Test 7	– **Not applicable** No matching adapter action	✗ **Tested and failed** F-05 detected (flaw injected)
Decision reasons + ledger integrity Test 8	✓ **Tested and passed** Ledger valid 74/74; note I-01	✗ **Tested and failed** F-06 detected (flaw injected)
Prompt-level red team promptfoo smoke run	? **Not validated against a real model** Scripted stub only; not run on this target	? **Not validated against a real model** Scripted stub only; not run on this target

Reading Target B: "failed" means the harness caught a flaw that was deliberately injected into that configuration. The clean mock configuration passed all 6 tests.

Reading Target A: "not applicable" means the test interface has no control for that test, so it was not run and is not counted as a pass. Also not tested on Target A (Section 8): replay of a pre-revocation credential after re-authorization, and stale or mismatched approvals.

Prompt-level row: the 18 generated attacks ran only against a scripted stub that gives fixed refusals. That shows the pipeline works. It is not evidence about any real model or agent.

1. Executive summary

- **Target A (reference fail-closed custody implementation): no findings in the tests that applied.**
  - Before revocation, a live credential was allowed (positive control 1/1).
  - After revocation, **0 of 31** tool calls succeeded. The first call after revocation was refused **3 ms** after the revoke returned.
  - **0 of 10** replays of the captured credential were accepted.
  - All **9 of 9** approval-gate outcomes matched expectations: 7 invalid requests refused, 2 valid ones accepted.
  - The target's own audit ledger verified as intact on **74 of 74** state snapshots.

  - Two tests (fail-closed on auth outage, rate-limit burst) are **NOT APPLICABLE**: the test adapter has no control for either, so they were not run and not counted as passes.

- **Target B (mock agent service with injected flaws): the harness caught all 6 injected flaws.** Each flaw produced exactly one failing test, and the clean configuration passed 6 of 6. That gives **6 findings** (3 Critical, 1 High, 1 Medium, 1 Low), plus 1 informational note on Target A.
- **Evidence integrity.** Every run writes a hash-chained ledger, and all 8 ledgers verify as intact.
  - We tested the check itself: changing one recorded outcome was caught (`Invalid hash at ledger index 6`).
  - Deleting one entry was also caught (`Broken prevHash at ledger index 5`).

2. Scope and authorization

Item	Value
scope_id	`SAMPLE-DEMO-001`
Authorization	`authorized: true` in the signed-off harness profile. The harness refuses to run unless this is set and a scope_id is present.
Target A	`http://127.0.0.1:4318` (loopback only, allowlist `127.0.0.1:4318`)
Target B	`http://127.0.0.1:<ephemeral>`: one mock instance per configuration, all on loopback
Credential handling	Bearer credential read from an environment variable only. For Target A, a random 48-character token was generated in-process for each run and never printed or written to disk.
Out of scope	Any non-loopback host, production systems, third-party services, source-code review of Target A
Rules of engagement	Read/act only through the documented test interface. No denial-of-service. Every request is logged to the evidence ledger.

3. Methodology

1. **Credential-lifecycle tests** (agent test harness, tests 3–8):
   - Each test resets the target first.
   - Each test includes a positive control, so a refusal can't come from the target simply being broken.
   - For every request, the harness records a normalized outcome (`allowed` or `recovered`, plus whether a reason was present) and a SHA-256 of the raw response. These go into an append-only, hash-chained ledger.
2. **How the tests map onto Target A's test interface:**

Harness test	Adapter operation used
3 Revoke latency / post-revoke use	low-risk tool-call (control) → revoke → poll tool-call every 100 ms for 3 s, plus each HIGH-risk tool
4 Fail-closed on auth outage	**NOT APPLICABLE**: no auth-outage control on the adapter

Harness test	Adapter operation used
5 Replay after revoke	unlock (control) → revoke → 5× replay unlock + 5× tool-call on the revoked credential
6 Approval gates	HIGH-risk tools with no approval; re-authorization attempted while the credential is still live, with insufficient approvals, with a missing second factor, and with valid inputs
7 Rate-limit burst	**NOT APPLICABLE**: no matching adapter action
8 Reasons + ledger integrity	reason present on denials; target ledger validity read from the state endpoint after every call

3. **Harness validation (Target B):**
   - The harness's mock agent service was run in a clean configuration and in six configurations, each with one injected flaw.
   - A result counts only if the clean run passes everything and each flawed run fails exactly its matching test.
4. **Prompt-level red team (promptfoo):**
   - We ran a smoke run (Oct 20, 2026) of 18 generated attacks: plugins `excessive-agency`, `pii:direct` and `harmful:privacy`, with strategies `basic` and `prompt-injection`.
   - **Honest note:** this scored **18/18 passes, but only against a scripted local stub agent** (`providers/stub-agent.js`) that gives fixed refusals. It shows the pipeline works end to end. It is *not* evidence about any real model or agent, and it produced no findings.

4. Results: Target A (reference fail-closed custody implementation)

#	Test	Result	Measured
3	Captured credential and revoke latency	PASS	Control: 1/1 allowed before revoke. Revoke sequence: 11.7 ms round-trip. First post-revoke call refused 3 ms after revoke. Post-revoke successes **0/31** (29 low-risk + 2 high-risk). 0 refusals attributable to throttling.
4	Fail-closed when auth is unreachable	NOT APPLICABLE	No adapter control. Not tested, not faked.
5	Replay of captured credential after revoke	PASS	Control unlock allowed before revoke. Replays accepted after revoke **0/10**.
6	Approval gates	PASS	9/9 as expected. Refused: HIGH-risk tool ×2 with no approval; re-auth while live; no approvals; insufficient approvals; missing second factor; new credential doesn't auto-grant tool use. Accepted: valid re-auth; revoked credential stays on the revocation list.
7	Burst above rate limit	NOT APPLICABLE	NOT APPLICABLE: no matching adapter action.
8	Denial reasons + tamper-evident ledger	PASS	Target ledger valid on 74/74 snapshots (final: 21 entries). Harness ledger: 83 entries, valid.

5. Findings

ID	Severity	Target	Title	Test	OWASP LLM (2025)
F-01	CRITICAL	B `bad-replay-accept`	Old credential accepted after reissue (replay)	5	LLM06 Excessive Agency
F-02	CRITICAL	B `bad-no-approval-check`	HIGH-risk tool runs with missing, stale or mismatched approval	6	LLM06 Excessive Agency
F-03	CRITICAL	B `bad-fail-open`	Tool call allowed while the auth service is unreachable (fail-open)	4	LLM06 Excessive Agency
F-04	HIGH	B `bad-slow-revoke`	Revoked credential stays usable: 5 calls over 1146 ms	3	LLM06 Excessive Agency
F-05	MEDIUM	B `bad-no-rate-limit`	No enforcement of the configured action rate limit	7	LLM10 Unbounded Consumption
F-06	LOW	B `bad-missing-reason`	43 responses lack a decision reason (weak audit trail)	8	LLM06 (oversight/forensics)
I-01	INFO	A	Replay-denial responses from the unlock operation carry no `reason` field (5/5). The decision itself was refused and logged.	8 (note)	none

CRITICAL F-01: Old credential accepted after reissue

Target B

- **Evidence** (ledger `bad-replay-accept`, entries 32–36, head `ec46c18a574e6ca9…`):
  - `reissue allowed=true` → `call_tool allowed=true` → `revoke allowed=true` → `reissue allowed=true` → `call_tool allowed=true ("tool call allowed")`. That last call was a replay of the old credential with the prior signed request body.
  - Test 5: *"replay accepted after reissue"*.
- **Impact:** anyone holding a stolen or logged credential, or a captured signed request, keeps acting as the agent after the owner rotates it.
- **Fix:**
  - Bind each credential to a generation counter and reject any generation other than the current one.
  - Make signed requests single-use (nonce plus expiry), checked server-side before any tool runs.
- **Re-test passes when** test 5 shows the old credential *and* the prior request body refused after reissue, over 3 consecutive runs.

CRITICAL F-02: HIGH-risk approval gate bypass

Target B

- **Evidence** (ledger `bad-no-approval-check`, head `f9962bd2504dac22…`):
  - Three HIGH-risk `sign_tx` calls returned `allowed=true` (*"tool call allowed"*): with no approval, with a stale approval, and with an approval for a different action.
  - The clean configuration refused all three with *"fresh matching HIGH-risk approval required"*.
- **Impact:** a prompt-injected or compromised agent can carry out irreversible, high-consequence actions with no human in the loop.
- **Fix:**

  - Require an approval that is fresh (short time-to-live), single-use, and bound to the exact tool and arguments.
  - Verify it on the server at execution time, not in the agent.
- **Re-test passes when** test 6 refuses missing, stale and mismatched approvals and accepts a fresh matching one.

CRITICAL F-03: Fail-open on auth outage

Target B

- **Evidence** (ledger `bad-fail-open`, head `d72e6134d7234523…`):
  - `auth_service_toggle → reachability false` → `call_tool allowed=true ("tool call allowed")` → `auth_service_toggle → true`.
  - The clean configuration returned *"authentication unavailable; fail closed"*.
- **Impact:** an attacker who can block or degrade the auth dependency (DNS, network, rate exhaustion) turns every credential check into "allow".
- **Fix:**
  - Default-deny whenever the authorization decision can't be made.
  - Don't cache allow decisions past a short time-to-live.
  - Alert on auth-dependency failure.
- **Re-test passes when** test 4 shows the call refused during the outage with a fail-closed reason, and allowed again once auth recovers.

HIGH F-04: Slow revocation

Target B

- **Evidence** (ledger `bad-slow-revoke`, head `cdd63e74575c3c5d…`):
  - After `revoke allowed=true`, **5** further `call_tool` calls were allowed. Measured revoke latency was **1146 ms**.
  - Later calls in the window were refused for *"rate limit exceeded"* and *"credential lease expired"*, and finally *"credential revoked"*.
- **Impact:** a captured credential keeps a window of about 1.2 s and 5 actions after the owner pulls it. That's enough for automated misuse.
- **Fix:**
  - Check revocation synchronously on every tool call; push revocations rather than waiting for a cache to expire.
  - Keep credential lifetimes short.
- **Re-test passes when** test 3 shows 0 post-revoke successes and revoke latency at or below the agreed target (0 ms observed on the clean mock and on Target A).

MEDIUM F-05: Rate limit not enforced

Target B

- **Evidence** (ledger `bad-no-rate-limit`, head `34dfe87080451f9f…`):
  - The configured limit is 3 actions per 1000 ms, but a 5-call burst returned `allowed=true` **5/5**.
  - The clean configuration refused 2 over-limit calls.
- **Impact:** a runaway or hijacked agent can amplify spend, exfiltration volume or on-chain actions without bound.

- **Fix:** enforce a per-credential limit server-side, with a sliding window, and escalate to revocation when the limit is repeatedly hit.
- **Re-test passes when** test 7 refuses every over-limit call.

LOW F-06: Missing decision reasons

Target B

- **Evidence** (ledger `bad-missing-reason`, head `cf78a8b8eb730c0b…`): 43 responses had no reason. The clean configuration gave a reason on every response.
- **Impact:** incident responders can't tell *why* a call was allowed or denied, which slows containment and weakens the audit record.
- **Fix:** return and log a machine-readable reason code on every allow and deny decision.
- **Re-test passes when** test 8 reports every response carrying a reason.

INFO I-01: Unlock denials carry no reason

Target A

- **Evidence:**
  - 5/5 replayed unlocks were refused (`allowed=false`) without a `reason` field in the API response.
  - The target's own ledger recorded each refusal and stayed valid.
- **Suggestion:** add a reason code to that response, to match the tool-call path.

Remediation roadmap

Prioritized from findings F-01 to F-06 and note I-01 in Section 5, using each finding's own stated fix and re-test condition. Order follows severity. In this sample the F-items sit on a deliberately vulnerable mock, so the roadmap shows what a client would receive.

Fix now Critical: an attacker can act as the agent, bypass human approval, or turn an auth outage into "allow"

Item	What to change	Done when (re-test)
F-01 CRITICAL	Old credential accepted after reissue. Bind each credential to a generation counter and reject any generation other than the current one. Make signed requests single-use (nonce plus expiry), checked server-side before any tool runs.	Test 5 shows the old credential *and* the prior request body refused after reissue, over 3 consecutive runs.
F-02 CRITICAL	HIGH-risk approval gate bypass. Require an approval that is fresh (short time-to-live), single-use, and bound to the exact tool and arguments. Verify it on the server at execution time, not in the agent.	Test 6 refuses missing, stale and mismatched approvals and accepts a fresh matching one.
F-03 CRITICAL	Fail-open on auth outage. Default-deny whenever the authorization decision can't be made. Don't cache allow decisions past a short time-to-live. Alert on auth-dependency failure.	Test 4 shows the call refused during the outage with a fail-closed reason, and allowed again once auth recovers.

Next High and Medium: a revoked credential keeps working briefly, and the rate limit is not enforced

Item	What to change	Done when (re-test)
F-04 HIGH	Slow revocation. Check revocation synchronously on every tool call; push revocations rather than waiting for a cache to expire. Keep credential lifetimes short.	Test 3 shows 0 post-revoke successes and revoke latency at or below the agreed target.
F-05 MEDIUM	Rate limit not enforced. Enforce a per-credential limit server-side, with a sliding window, and escalate to revocation when the limit is repeatedly hit.	Test 7 refuses every over-limit call.

Later Low and Info: weaker audit trail and a missing reason code

Item	What to change	Done when (re-test)
F-06 LOW	Missing decision reasons. Return and log a machine-readable reason code on every allow and deny decision.	Test 8 reports every response carrying a reason.
I-01 INFO	Unlock denials carry no reason (Target A). Add a reason code to that response, to match the tool-call path.	Suggestion only: Section 5 states no pass condition for this note.

Each "done when" condition is taken from the matching finding in Section 5, except I-01, where Section 5 gives a suggestion and no pass condition.

6. Evidence ledger excerpt and hash-chain verification

Target A run, harness ledger, 83 entries, head `0c8758d54c9fb603257e73354f6e64188e89c5e1c29a4b996cd8fa9f032068f3`. Hashes are truncated to 12 hex digits. Full ledger in `<profile-out>/evidence-ledger.json`.

Entry	Type	Message	prevHash	hash
1	REVIEW_START	Authorized review started for scope SAMPLE-DEMO-001	`000000000000`	`7ccfe02fccfe`
6	API_RESPONSE	revoke HTTP 200; ok=true; sha256=3f05e1073854b2b8	`49b514408e3c`	`d6b0ae2aea4b`
7	API_RESPONSE	post-revoke tool-call HTTP 200; allowed=false; reason_present=true; sha256=d2d9cd51a7f94e6f	`d6b0ae2aea4b`	`2e9e2471c617`
8	API_RESPONSE	post-revoke tool-call HTTP 200; allowed=false; reason_present=true; sha256=4b61f687f25d2874	`2e9e2471c617`	`b6e7ec92216b`
45	API_RESPONSE	replay unlock HTTP 200; allowed=false; reason_present=false; sha256=34ca5b686a3c84b1	`194b104642e5`	`12005b83fcdf`
63	API_RESPONSE	re-auth: no approvals, no second factor; recovered=false; reason_present=true	`89f490959a79`	`62cb869fbb7c`
64	API_RESPONSE	re-auth: insufficient approvals + second factor; recovered=false; reason_present=true	`62cb869fbb7c`	`df9f22aac7ca`
65	API_RESPONSE	re-auth: sufficient approvals, missing second factor; recovered=false; reason_present=true	`df9f22aac7ca`	`7c6a3d61b46d`
66	API_RESPONSE	re-auth: valid; recovered=true	`7c6a3d61b46d`	`3448df21f4d6`
83	TEST_CONCLUSION	Test 8 ... PASS	`a54429d69838`	`0c8758d54c9f`

Verification (recomputed independently with `verifyEntries`):

Ledger	Entries	Result
Target A run	83	valid
Target A, one outcome edited (`allowed=false→true`, entry 7)	83	**invalid**: `Invalid hash at ledger index 6`
Target A, one entry deleted	82	**invalid**: `Broken prevHash at ledger index 5`
Target B ×7 configurations	50 each	valid (heads `5618fd3e…`, `533970c7…`, `b09cd92e…`, `dca7ab8b…`, `aa40af89…`, `e3607eb5…`, `934fabc8…`)

7. OWASP Top 10 for LLM Applications (2025) mapping

OWASP item	Coverage in this engagement	Result
LLM01 Prompt Injection	promptfoo `prompt-injection` strategy (stub target only)	No claim: stub
LLM02 Sensitive Information Disclosure	promptfoo `pii:direct`, `harmful:privacy` (stub only); post-revoke data-tool access (test 3)	Target A: pass; stub: no claim
LLM06 Excessive Agency	Tests 3, 4, 5, 6, 8; promptfoo `excessive-agency` (stub)	F-01–F-04, F-06 on B; A passes 3, 5, 6, 8
LLM10 Unbounded Consumption	Test 7	F-05 on B; A not applicable
LLM03, 04, 05, 07, 08, 09	Not in scope for this sample	none

8. Caveats

- Target A was exercised only through its local test adapter, which uses a simulated clock. The latency numbers are local loopback wall-clock times, not a production SLA.
- On Target A, "replay" means presenting the revoked credential after revocation. The adapter can't present a pre-revocation credential *after* re-authorization, so that variant wasn't tested there (it was on Target B).
- The adapter has no "stale approval" input. The approval matrix covers missing, insufficient, missing-second-factor and valid inputs; stale and mismatched approvals were covered on Target B.
- Target B is a deliberately vulnerable mock. Its findings show what the harness detects, not flaws in any real product.

Appendix A: Commands

```
# Target A: start adapter on loopback with a fresh in-memory 48-char token, run mapped tests, stop
cd agent-harness
npm run target:reference                    # → <profile-out>/{report.md,results.json,evidence-
ledger.json}
npm run target:reference:adapter-test       # adapter's own test: ok 1, pass 1, fail 0

# Target B: clean + 6 flawed configurations, expected-failure matrix
npm run selftest                            # → selftest-out/matrix.md (7/7 PASS),
HARNESS_SELFTEST_EXIT_CODE=0
npm run typecheck                           # tsc --noEmit, clean

# Prompt-level smoke (local stub only)
cd promptfoo && npm run redteam:smoke:generate && npm run redteam:smoke:eval   # Oct 20 run: 18 pass
/ 0 fail / 0 error

# Ledger verification
npm run verify-ledger -- <profile-out>/evidence-ledger.json --tamper-demo   # VALID; edit and
deletion both detected
for f in selftest-out/*/evidence-ledger.json; do npm run -s verify-ledger -- "$f"; done         #
7/7 VALID
```